

Universidad CENFOTEC

Escuela de Software

Curso: Estructura de Datos

Docente: Romario Salas Cerdas

Estudiante: Santiago Hernandez

Proyecto: Primer Estudio de Caso

Fecha: 17 de octubre de 2025

Aplicación de la estructura de pila en la funcionalidad “Cadenas de impresión”

1. ¿Por qué una pila es la estructura de datos adecuada?

La funcionalidad denominada “*Cadenas de impresión*” busca simular el proceso mediante el cual un programa construye mensajes o salidas de texto a partir de distintos fragmentos (tokens). Estos tokens pueden representar constantes literales, variables, operadores de concatenación y especificadores de formato.

En este contexto, la estructura de datos más adecuada es una **pila (stack)**, ya que su comportamiento **LIFO (Last In, First Out)** garantiza que los fragmentos se procesen de manera ordenada y predecible, respetando la secuencia en que fueron apilados.

Cuando un usuario agrega fragmentos de texto o variables, estos se van almacenando en la pila conforme se introducen. Al generar la cadena final, los elementos se recorren en sentido inverso (del tope a la base), reproduciendo el orden exacto de ingreso y asegurando que la composición final sea coherente. Este enfoque refleja cómo los lenguajes de programación y sistemas operativos manejan internamente las llamadas de impresión, los búferes de salida o los concatenadores de texto.

Una lista o arreglo no sería tan eficiente ni intuitivo para esta tarea, ya que implicaría desplazar índices o recorrer todo el conjunto para acceder al último elemento agregado. En cambio, la pila permite un **acceso inmediato al elemento más reciente**, que en este caso corresponde al último fragmento de texto o formato añadido.

De este modo, la pila no solo simplifica la implementación, sino que **modela de manera natural el proceso de ensamblaje de cadenas**.

2. Seguridad, orden y eficiencia que aporta la pila

El uso de una pila en la funcionalidad de cadenas de impresión proporciona **seguridad**, **orden** y **eficiencia**, tres principios clave en el diseño de estructuras de software:

- **Seguridad:**

La pila asegura la integridad del orden de los tokens. Cada elemento es insertado mediante una operación `push()` y eliminado (o leído) mediante `pop()` o `peek()`. Este control evita desbordamientos o manipulaciones fuera del flujo natural de inserción y extracción, reduciendo errores lógicos. Además, el usuario no accede directamente a posiciones arbitrarias; la estructura controla el acceso de manera encapsulada.

- **Orden:**

El comportamiento LIFO garantiza que los fragmentos de texto se ensamblen exactamente en el orden en que fueron introducidos. El primer fragmento agregado termina al inicio de la cadena final, mientras que el último se coloca al final. Esto asegura una **reconstrucción ordenada y coherente**, especialmente cuando se manejan varios tipos de tokens, como literales, variables y formatos.

- **Eficiencia:**

Las operaciones fundamentales de una pila (`push()` y `pop()`) se ejecutan en **tiempo constante $O(1)$** , sin necesidad de recorrer la estructura completa. La concatenación final se realiza mediante un único recorrido lineal, manteniendo un consumo de memoria bajo y un desempeño óptimo incluso con gran cantidad de tokens. Esta eficiencia la convierte en una estructura ideal para sistemas que requieren manipular textos dinámicos o buffers de salida.

En conjunto, estas propiedades hacen que la pila proporcione un entorno controlado, predecible y de bajo costo computacional para la construcción de cadenas de texto, cumpliendo con las exigencias de seguridad, orden y rendimiento.

3. Ejemplo concreto en software de sistema

Caso de prueba: Inclusión de formato de tabulación

Descripción:

Se evalúa si la pila mantiene correctamente el orden de tokens al incluir un formato de tabulación (\t) entre variables y textos literales.

Entradas:

1. Constante literal → "Nombre:"
2. Operador → "+"
3. Variable → "%s"
4. Operador → "+"
5. Formato → "\\t"
6. Operador → "+"
7. Constante literal → "Edad:"
8. Operador → "+"
9. Variable → "%s"

Salidas:

```
=====
      PILA DE CADENAS DE IMPRESIÓN
=====
1. Agregar nuevo token
2. Mostrar pila actual
3. Generar e imprimir cadena final
0. Salir
-----
Seleccione una opción: 3

Resultado final de concatenación:

"Nombre:"[variable] "Edad:"[variable]
```

Análisis:

La salida confirma el comportamiento LIFO.

La concatenación respeta el orden de ingreso y los operadores + actúan de forma implícita.

El formato \t se traduce correctamente en una tabulación entre ambos bloques de texto.

Conclusión

La implementación de la estructura de pila en la funcionalidad “*Cadenas de impresión*” permitió evidenciar de forma práctica cómo una estructura simple, basada en el principio LIFO (Last In, First Out), puede organizar y gestionar eficientemente distintos fragmentos de texto antes de generar una salida final.

El uso de tokens diferenciados —constantes literales, variables, operadores de concatenación y formatos— demostró que la pila aporta **orden, seguridad y eficiencia** al proceso de construcción de cadenas. Su comportamiento garantiza que cada elemento se procese en el orden correcto, evitando errores y reduciendo la complejidad del manejo de datos.

En conclusión, este estudio de caso confirma que las pilas no solo son esenciales en estructuras de control y ejecución, sino que también tienen aplicaciones directas en la manipulación y formateo de información en software real, siendo una solución clara, estructurada y confiable para este tipo de tareas.

Repository's link: <https://github.com/sant1agozs/EDEstudioDeCaso>